[GURU GHASIDAS VISHWAVIDYALAYA]

[INFORMATION TECHNOLOGY]

PROJECT REPORT

ON

Hotel Management System: A Console-Based Room Booking, Food Ordering, Billing and Staff Management Application Using Object-Oriented C++

Submitted in partial fulfilment of the requirements for

Object-Oriented Programming in C++ Laboratory

Submitted by

1. [Vaibhav Kumar] — Roll No. [25008289]
2. [Shreyansh Tiwari] — Roll No. [25008282]

Under the guidance of

[Asst. Prof. Suhel Ahamed]

Academic Year: [2026–27]

Date: [29/10/2026]

1. CERTIFICATE PAGE

CERTIFICATE

This is to certify that the project report entitled **"Hotel Management System: A Console-Based Room Booking, Food Ordering, Billing and Staff Management Application Using Object-Oriented C++"** is a bonafide work carried out by **Vaibhav Kumar (Roll No. 25008289)** and **Shreyansh Tiwari (Roll No. 25008282)** of the Department of Information Technology, Guru Ghasidas Vishwavidyalaya, Bilaspur, in partial fulfilment of the requirements of the **Object-Oriented Programming in C++ Laboratory** during the academic year 2026–27.

The work embodied in this report has been carried out under my guidance and supervision. The project has been tested by the students and, to the best of my knowledge, the matter presented here is original and has not been submitted elsewhere for any other degree, diploma or certificate.

Project Guide
Asst. Prof. Suhel Ahamed
Department of Information Technology

Signature: _____

Date: _____

Head of Department
Department of Information Technology
Guru Ghasidas Vishwavidyalaya

Signature: _____

Date: _____

Submitted by:

1. Vaibhav Kumar — 25008289
2. Shreyansh Tiwari — 25008282

2. INDEX

S. No.	Section	Page No.
1	Certificate Page	[2]
2	Index	[3]
3	Introduction	[4]
4	Motivation	[6]
5	Design	[7]
6	Implementation	[12]
7	Result and GitHub link	[17]
8	Conclusion	[18]
9	Scope and Limitations	[19]
10	References	[20]
11	Appendix	[21]

Sub-sections covered in the report:

- **Introduction** – overview, problem statement, objectives, scope, technologies used, report organisation.
- **Motivation** – problems of manual hotel management, reason for choosing the project and the language.
- **Design** – architecture, class diagrams, class descriptions, file formats, program flow, menus and OOPS mapping.
- **Implementation** – how each class and function works, with code excerpts.
- **Result** – sample outputs, test cases and observations.
- **Conclusion**, **Scope and Limitations**, **References** and **Appendix** (build steps and source excerpts).

3. INTRODUCTION

3.1 Overview

Hotels deal every day with a large amount of information: which rooms are free, who has booked which room and for how many days, what food has been ordered, how much each guest has to pay, which employees are on duty and how much they should be paid at the end of the month. When these records are written in registers, mistakes, duplicate entries and lost information are common, and finding a single record can take a long time.

The **Hotel Management System** developed in this project is a console-based application written in **C++** that automates these activities. The program offers two kinds of accounts. The **Admin** (the hotel manager) can view and add rooms, add food items, change prices, order food for a room, generate a bill and check a guest out, view the total revenue and manage staff. A **Customer** can register, log in, view rooms, book an available room, order food during the stay, view their own bookings and finally view the bill and check out.

The application is built on the principles of **Object-Oriented Programming (OOP)**. Every real-world object in the hotel, such as a room, a food item, a customer or a staff member, is represented by a class. The classes hide their data, expose only safe operations, and are connected through inheritance and aggregation. All information is stored permanently in plain text files, so the data is still available when the program is closed and started again.

3.2 Problem Statement

Small and medium hotels often depend on paper registers or on general-purpose spreadsheets for their daily work. These methods are slow, difficult to search, easy to damage and do not prevent mistakes such as booking the same room twice or charging a guest the wrong amount. The problem addressed by this project is to design and implement a simple, reliable and easy-to-use software system that manages rooms, bookings, food orders, billing, revenue and staff records, using object-oriented techniques and permanent file storage, without needing any database server or internet connection.

3.3 Objectives

- To model the main entities of a hotel (room, food item, customer, administrator, staff member) as C++ classes.
- To provide separate, role-based dashboards for the Admin and for Customers.
- To prevent invalid operations such as double booking, negative prices, empty names and invalid numeric input.
- To calculate the bill automatically from the room price, the number of days and the food ordered.
- To keep track of room revenue and food revenue.
- To manage staff details, daily attendance and salary payments for a given month.

- To store all data in files so that it remains available between program runs.
- To apply and demonstrate the major OOP concepts: encapsulation, abstraction, inheritance, polymorphism, overloading, templates, exception handling and file handling.

3.4 Users of the System

User	Role in the system	Main operations
Admin	Hotel manager with full control. Uses a fixed admin account.	View rooms and bookings, add rooms and food, change prices, order food for a room, generate bill and check out, view revenue, manage staff.
Customer	Guest who registers an account (username, password and Aadhaar number).	View rooms, book a room, view own bookings, order food, view bill and check out.

3.5 Technologies and Tools Used

Item	Details
Programming language	C++ (object-oriented, compiled; C++11 features such as move semantics, auto, range-based for loops and brace initialisation are used)
Libraries (STL)	iostream, fstream, sstream, string, vector, iomanip, limits, stdexcept, algorithm, utility
Data storage	Plain text files with the '|' character as field separator: rooms.txt, foods.txt, customers.txt, bookings.txt, staff.txt, attendance.txt, payments.txt
User interface	Menu-driven console (terminal) interface
Compiler / IDE	Any standard C++ compiler such as g++ (MinGW), Code::Blocks, Dev-C++ or Visual Studio

3.6 Organisation of the Report

The report follows the order of the index. Section 4 explains why this project was chosen. Section 5 describes the design: architecture, class diagrams, data files, program flow and menus. Section 6 explains the implementation of the important classes and functions. Section 7 presents sample results and test cases. Section 8 gives the conclusion, Section 9 discusses scope and limitations, Section 10 lists the references and Section 11 (Appendix) contains build instructions and selected source code.

4. MOTIVATION

4.1 Problems of Manual Hotel Management

- **Double booking:** in a paper register it is easy to give the same room to two guests.
- **Calculation errors:** bills that combine room rent, number of days and several food orders are often added up by hand and may contain mistakes.
- **Slow searching:** finding which guest is in which room, or which rooms are free, requires going through many pages.
- **No permanent, organised record:** registers can be lost, damaged or become unreadable.
- **Staff records:** attendance and salary calculations are time-consuming and a staff member can accidentally be paid twice for the same month.
- **No role separation:** any person with access to the register can see and change everything.

4.2 Why a Hotel Management System?

A hotel is an excellent real-world example for learning object-oriented programming because it is full of natural objects. A room has a number, a type, a price and a booking status; a customer has a username, password and identity number; a staff member has a role and a wage. These objects interact with each other in clearly defined ways: a customer books a room, a room stores the food ordered by its guest, and the hotel keeps the complete collection of rooms, customers and staff. This one-to-one match between real-world objects and C++ classes makes the design easy to understand and also makes it possible to demonstrate almost every topic of the OOP laboratory in a single meaningful project.

4.3 Why C++ with File Handling?

C++ supports classes, inheritance, virtual functions, operator and function overloading, templates and exception handling, and it also gives good control over memory and performance. The Standard Template Library provides ready-made containers such as **vector** and utilities such as **stringstream**, which reduce the amount of code that has to be written. File handling with **ifstream** and **ofstream** was chosen for storage because it works on every computer without installing a database, and it lets students clearly see how data is converted to text, written to a file and read back.

4.4 Learning Outcomes

- Designing a program as a set of cooperating classes instead of one long procedure.
- Protecting data using access specifiers and validating it inside constructors and setter functions.
- Reusing code through inheritance and achieving different behaviour through virtual functions.
- Writing robust input routines and handling errors with exceptions.

5. DESIGN

5.1 Design Approach

The system was designed using an **object-oriented, modular approach**. First the entities of the hotel were identified; then each entity was turned into a class with its own data and operations; finally a controller class, **Hotel**, was created to hold the collections of objects and to provide the high-level operations used by the menus. The responsibilities are divided so that each class does one clear job: Room knows how to book and release itself, FoodItem knows its name and price, User and its children know how to log in and which menu to show, and Hotel knows how to search, save, load and bill.

5.2 System Architecture

The program can be viewed as three layers. The **presentation layer** consists of the menus and the input functions. The **logic layer** consists of the classes that enforce the rules of the hotel. The **data layer** consists of the text files in which the information is stored.

Layer	Components	Responsibility
Presentation	dashboard() menus, readInt(), readDouble(), readLine(), showRooms(), showBookings()	Displays menus and results; reads and validates user input.
Logic	Room, FoodItem, User, Admin, Customer, Staff, Attendance, Payment, Hotel	Implements booking rules, food ordering, billing, revenue, attendance and salary rules; throws exceptions for invalid actions.
Data	rooms.txt, foods.txt, customers.txt, bookings.txt, staff.txt, attendance.txt, payments.txt	Permanent storage; loaded in loadData() and written by the save functions.

5.3 Class Diagram

Figure 5.1 shows the inheritance relationship between the user classes. **User** is an abstract base class (it contains the pure virtual function *dashboard()*), so it cannot be instantiated. **Admin** and **Customer** inherit from it and override *dashboard()* to display their own menus.

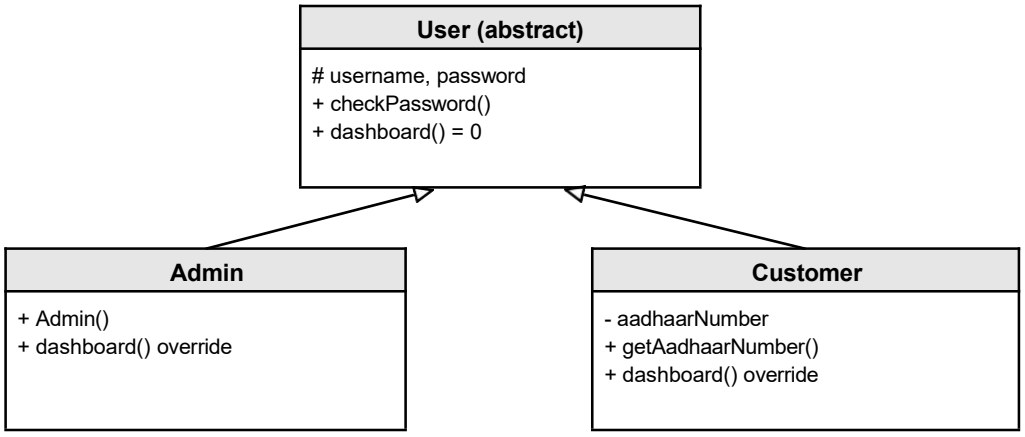


Figure 5.1: Inheritance hierarchy of the user classes

Figure 5.2 shows the **Hotel** class, which aggregates the other classes. Hotel owns vectors of Room, FoodItem, Customer, Staff, Attendance and Payment objects and also stores the running totals of room revenue and food revenue.

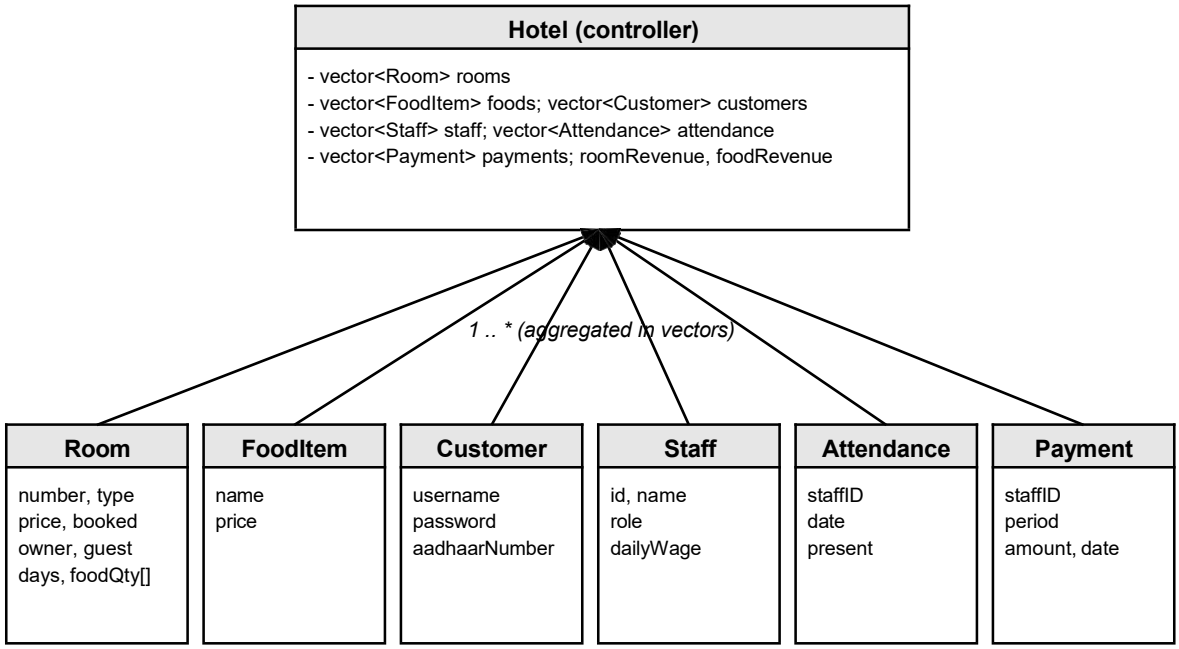


Figure 5.2: Hotel class and the objects it manages

5.4 Description of Classes

Class / Structure	Important data members	Important member functions
Room	number, type, price, booked, owner, guest, days, foodQty (vector of 100 ints), static objectCount	book(), addFood(), release(), restoreBooking(), setPrice(), getters, static getObjectCount(), friend operator<<
FoodItem	name, price	getName(), getPrice(), setPrice()

Class / Structure	Important data members	Important member functions
User (abstract)	username, password (protected)	getUsername(), checkPassword(), getPasswordForStorage(), pure virtual dashboard()
Admin	Fixed username and password set in the constructor	dashboard() with 11 options
Customer	aadhaarNumber (in addition to inherited data)	getAadhaarNumber(), dashboard() with 7 options
Staff	id, name, role, dailyWage	getID(), getName(), getRole(), getDailyWage()
Attendance (struct)	staffID, date, present	–
Payment (struct)	staffID, period, amount, date	–
Hotel	rooms, foods, customers, staff, attendance, payments, roomRevenue, foodRevenue	findRoom() (overloaded), findStaff(), userExists(), paidFor(), presentDays(), absentDays(), save and load functions, showRooms(), showBookings()

5.5 Data Design: File Formats

Each type of record is stored in its own text file with one record per line. The fields of a record are separated by the vertical bar character '|'. To make sure that a user cannot break this format by typing the separator inside a name, the function *cleanField()* replaces every '|' with '/' before the data is stored.

File	Record format	Meaning
rooms.txt	number|type|price	One line per room
foods.txt	name|price	One line per menu item
customers.txt	username|password|aadhaar	One line per registered customer (older records without Aadhaar are still accepted)
bookings.txt	roomNo|owner|guest|days|qty0|qty1|...	One line per currently booked room; the quantities are the food ordered, in menu order
staff.txt	id|name|role|dailyWage	One line per staff member
attendance.txt	staffID|date|present	present is 1 for present and 0 for absent
payments.txt	staffID|period|amount|date	One line per salary payment; period is in YYYY-MM form

5.6 Program Flow

When the program starts it first loads all the data files. If the room file or the food file is empty or missing, default rooms and a default menu are created and saved. The user then registers or logs in, and the program shows the dashboard that matches the role of the user. After every operation that changes data, the corresponding file is saved again so that nothing is lost if the program is closed.

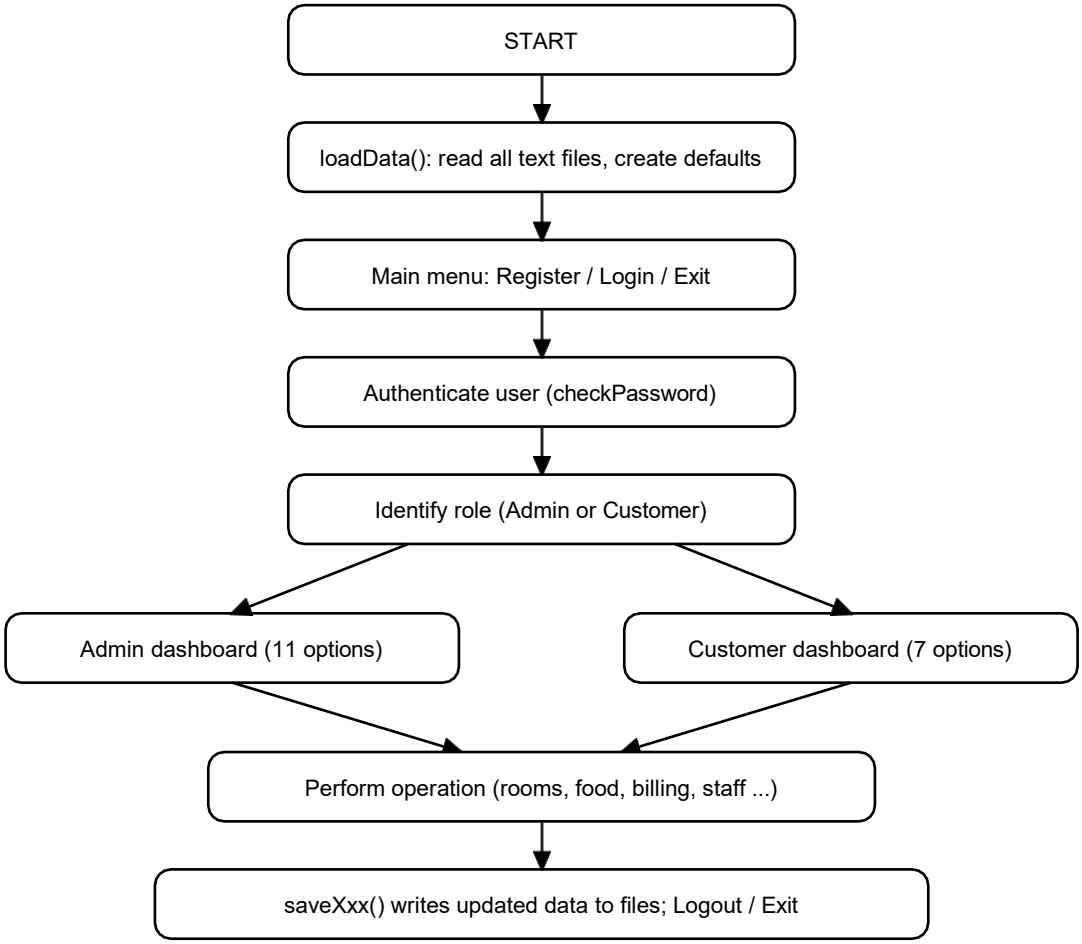


Figure 5.3: Overall flow of the program

5.7 Menu Design

Admin dashboard	Customer dashboard
1. View all rooms 2. View available rooms 3. View all active bookings 4. Add new room 5. Add new food item 6. Change room/food prices 7. Order food for a room 8. Generate bill and checkout 9. View total revenue 10. Staff management 11. Logout	1. View all rooms 2. View available rooms 3. Book a room 4. View my bookings 5. Order food 6. View bill and checkout 7. Logout

5.8 Mapping of OOP Concepts to the Design

OOP concept	Where it is used
Class and object	Room, FoodItem, Customer, Staff, Hotel and the vectors of objects held by Hotel
Encapsulation	Private data members with getters and validating setters (for example Room::setPrice())
Abstraction	Abstract class User; the Hotel class hides file handling and searching from the menus
Inheritance	Admin and Customer derive from User
Runtime polymorphism	Virtual function dashboard() overridden in Admin and Customer
Function overloading	Hotel::findRoom(int) and Hotel::findRoom(const string&)
Operator overloading / friend function	operator<< for Room
Constructors, copy and move	Default, parameterised, copy and move constructors and destructor of Room
Static member	Room::objectCount and Room::getObjectCount()
Template	showMessage(const T&)
Exception handling	invalid_argument and runtime_error thrown by constructors and member functions
File handling	ifstream / ofstream in the load and save functions

5.9 Design Decisions

- **Text files instead of a database:** keeps the project self-contained and shows the basics of file I/O.
- **Food quantities stored inside Room:** a booking and its food orders live and die together, so when a room is released the food counts are cleared automatically.
- **Index-based food quantities:** the position of a food item in the menu is its key, so the booking file only needs a list of numbers.
- **Validation inside classes:** constructors and setters throw exceptions, so invalid objects can never be created, regardless of which menu option calls them.
- **Separate Admin and Customer classes:** allows each role to have its own menu through polymorphism instead of long if-else chains.

6. IMPLEMENTATION

6.1 Development Environment

The whole project is written in a single C++ source file. The program is compiled with a standard compiler, for example *g++ hotel.cpp -o hotel -std=c++11*, and run in a terminal. The file begins with the required header files and the statement *using namespace std;*. The names of all data files are stored in global constant strings (ROOMS_FILE, FOODS_FILE, CUSTOMERS_FILE, BOOKINGS_FILE, STAFF_FILE, ATTENDANCE_FILE and PAYMENTS_FILE), so a file name has to be changed in only one place.

6.2 Console Input Functions

Console programs easily crash or loop forever when the user types letters where a number is expected. To prevent this, the project uses three small helper functions. **readLine()** displays a prompt and reads a full line (using *getline(cin >> ws, value)* so that spaces inside names are kept). **readInt()** and **readDouble()** repeat the question until the user enters a valid number; when the stream enters an error state, the error is cleared with *cin.clear()* and the bad characters are discarded with *cin.ignore()*. readDouble() additionally insists on a value greater than zero, which is suitable for prices and wages.

```
int readInt(const string& prompt)
    { while (true) {
        cout << prompt;
        int value;
        if (cin >> value)
            { cin.ignore(numeric_limits<streamsize>::max(),
            '\n'); return value;
        }
        cout << "Invalid input. Enter a whole number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
```

6.3 Utility Functions: cleanField() and split()

Because every record is saved as one text line, two helper functions are needed. **cleanField()** uses the STL algorithm *replace()* to change the separator character '|' into '/' in any text entered by the user. **split()** takes a line and a delimiter, wraps the line in a *stringstream* and repeatedly calls *getline(ss, item, delim)* to collect the fields in a vector of strings. These two functions are used by all constructors that take text and by every loading routine.

6.4 The Room Class

The **Room** class is the most important class of the project. It stores the room number, type and price, and also the complete state of the current booking: whether the room is booked, the username of the customer who owns the booking, the guest's name, the number of days and a *vector<int> foodQty* that holds how many units of each menu item have been ordered. All data members are private.

The parameterised constructor validates the data and throws an *invalid_argument* exception if the room number or price is not positive or the type is empty. The class also defines a **copy constructor**, a **move constructor** and a **destructor**. A **static** data member, *objectCount*, is increased in every constructor and decreased in the destructor, so that *Room::getObjectCount()* always reports how many Room objects exist; this value is displayed under the room list.

The booking logic is inside **book()**. It throws a *runtime_error* if the room is already booked and an *invalid_argument* if the number of days is not positive; otherwise it marks the room as booked, stores the owner, guest and days, and sets all food quantities to zero. **addFood()** refuses to work on a room that is not booked and checks that the food index and the quantity are valid before increasing the count. **release()** clears every booking detail when the guest checks out, and **restoreBooking()** is used while loading bookings from the file.

```
void book(const string& user, const string& guestName, int stayDays)
    { if (booked) throw runtime_error("Room is already booked.");
    if (stayDays <= 0) throw invalid_argument("Days must be greater than zero.");
    booked = true;  owner = user;  guest = guestName;  days = stayDays;
    fill(foodQty.begin(), foodQty.end(), 0);
}

void addFood(size_t i, int quantity) {
    if (!booked) throw runtime_error("Room is not booked.");
    if (i >= foodQty.size() || quantity <= 0)
        throw invalid_argument("Invalid food quantity.");
    foodQty[i] += quantity;
}
```

6.5 The FoodItem Class

A **FoodItem** has a name and a price. Its constructor cleans the name with cleanField() and throws an exception when the name is empty or the price is not positive. A setter, *setPrice()*, is provided so that the admin can change the price later, and it applies the same positive-price rule. By default the menu contains Pizza (200), Burger (100), Sandwich (80), Biryani (180) and Coffee (50).

6.6 The User, Admin and Customer Classes

User is an abstract base class with the protected members *username* and *password*. It supplies *getUsername()*, *checkPassword()* and *getPasswordForStorage()*, and declares *virtual void dashboard() const = 0;*. It also has a virtual destructor so that objects can be deleted safely through a base-class pointer.

Admin calls the base constructor with the fixed account values and overrides dashboard() to print the eleven administrator options. **Customer** adds the private member *aadhaarNumber* with a getter and overrides dashboard() to print the seven customer options. Since both classes are used through the common interface of User, the program can ask any logged-in user to show its own menu without checking its type.

```
class User
{ protected:
    string username, password;
public:
    User(string u = "", string p = "") : username(std::move(u)), password(std::move(p)) {}
    virtual ~User() = default;
    bool checkPassword(const string& p) const { return password == p; }
    virtual void dashboard() const = 0;      // pure virtual: User is abstract
};
class Customer : public User { /* adds aadhaarNumber, overrides dashboard() */ };
```

6.7 Staff, Attendance and Payment

The **Staff** class stores the staff ID, name, role and daily wage and validates all of them in its constructor. Attendance and salary payments are simple records and are therefore written as **structures**: *Attendance* holds the staff ID, the date in YYYY-MM-DD form and a Boolean present flag, and *Payment* holds the staff ID, the pay period in YYYY-MM form, the amount and the payment date. Keeping these as plain structures makes them quick to create and easy to store in vectors.

6.8 The Hotel Class

Hotel is the controller of the system. Its private data members are the vectors of rooms, foods, customers, staff, attendance records and payments, together with *roomRevenue* and *foodRevenue*. Its private helper functions are:

- **findRoom(int number)** – returns the position of the room with the given number, or −1.
- **findRoom(const string& type)** – an overloaded version that returns the first *available* room of the given type, for example “AC”.
- **findStaff(int id)** – searches the staff list by ID.
- **userExists(username)** – prevents two customers from registering with the same username.
- **presentDays()** and **absentDays()** – count the attendance records of a staff member whose date begins with a given YYYY-MM period.
- **paidFor(id, period)** – checks whether salary has already been paid for that month.

The two versions of findRoom() are an example of **function overloading**: the same name is used, and the compiler selects the correct version from the type of the argument.

6.9 Saving and Loading Data

There is one **save** function for every file: saveRooms(), saveFoods(), saveCustomers(), saveBookings(), saveStaff(), saveAttendance() and savePayments(). Each opens an *ofstream*, throws a *runtime_error* if the file cannot be opened, and writes one record per line with ‘|’ between the fields. saveBookings() writes only rooms that are currently booked and appends the food quantity for every menu item.

The function **loadData()** reads the files in a fixed order: rooms, foods, customers, staff, attendance, payments and finally bookings. Bookings are read last because they need the rooms to exist already. For each line the record is split into fields and converted with *stoi()* or *stod()*

inside a *try/catch* block, so a damaged line is ignored instead of crashing the program. Duplicate room numbers and duplicate usernames are skipped. When no rooms or no food items are found, the default data is created and saved immediately.

```
ifstream rf(ROOMS_FILE);
while (getline(rf, line)) {
    auto v = split(line);
    if (v.size() >= 3) {
        try {
            int n = stoi(v[0]);  double p = stod(v[2]);
            if (findRoom(n) < 0) rooms.emplace_back(n, v[1], p);
        } catch (...) { /* ignore invalid records */ }
    }
}
if (rooms.empty()) { rooms.emplace_back(101, "Non-AC", 1000); /* ... */ saveRooms(); }
```

6.10 Displaying Rooms and Bookings

The stream insertion operator is overloaded as a **friend** function of Room, so a room can be printed with a single statement, *cout* << *r*. The operator uses *setw()*, *fixed* and *setprecision(2)* from <*iomanip*> to print the number, type, price and status (“Booked” or “Available”) in neat columns. **showRooms(bool availableOnly)** uses this operator to list either all rooms or only the free ones, prints “No matching rooms found.” when the list is empty, and finally prints the number of Room objects currently stored. **showBookings(admin, username)** prints the active bookings: an administrator sees every booking, while a customer sees only the bookings that belong to their own username.

6.11 Booking, Food Ordering, Billing and Checkout

Booking. The customer chooses a room number, enters the guest name and the number of days, and the menu code calls Room::book() with the logged-in username. Any exception thrown by the room (for example “Room is already booked.”) is caught and shown as a message, and the bookings file is saved after a successful booking.

Food ordering. The menu is displayed with the position and price of each item. The user selects an item and a quantity, and Room::addFood() adds the quantity to the counter at that position. The quantities are saved with the booking, so food ordered before the program is closed is not lost.

Billing and checkout. The bill of a stay is the sum of the room charge and the food charge: **room price × number of days** plus, for every menu item, **quantity × item price**. When the guest checks out, the room amount is added to *roomRevenue*, the food amount to *foodRevenue*, the room is released with Room::release() so that it becomes available again, and the files are saved. The administrator’s “View total revenue” option reports these totals.

6.12 Staff Management Workflow

- **Adding staff:** ID, name, role and daily wage are entered; findStaff() makes sure the ID is new; a Staff object is created and saveStaff() is called.

- **Marking attendance:** the admin gives the staff ID, the date and whether the person was present; an Attendance record is added to the vector and saveAttendance() is called.
- **Calculating salary:** for a chosen month, presentDays() counts the days present using *date.substr(0, 7)* to compare the year and month; the salary is the number of days present multiplied by the daily wage.
- **Recording payment:** paidFor() is checked first so a person cannot be paid twice for the same month; then a Payment record is stored and saved.

```
int presentDays(int id, const string& period) const
    { int count = 0;
    for (const auto& a : attendance)
        if (a.staffID == id && a.present &&
            a.date.size() >= 7 && a.date.substr(0, 7) == period)
            ++count;
    return count;
}
```

6.13 Exception Handling

Errors that come from invalid data are reported with C++ exceptions instead of return codes. Constructors and setters throw **invalid_argument** for illegal values (non-positive prices, empty names, an invalid food index). Operations that are not allowed in the current state, such as booking a booked room, ordering food for a room that is not booked, or failing to open a file for writing, throw **runtime_error**. The calling code catches these exceptions, shows the message to the user and continues, so a single mistake never ends the program.

6.14 Function Template

The template *template <typename T> void showMessage(const T& message)* prints a value followed by a new line. Because T is deduced from the argument, the same function can display a string, an integer or a floating-point number, which demonstrates **generic programming**.

6.15 Testing Approach

The program was tested module by module. Valid and invalid values were entered for every input, rooms were booked twice, food was ordered for free rooms, the program was closed and restarted to check persistence, and damaged lines were placed in the data files to see that they were ignored. The results are summarised in Section 7.

7. RESULT

The project was compiled and executed as a console program. The following illustrative outputs show what the user sees. They are based on the default data created by the program and the output format defined in the code.

7.1 Sample Output: Room List

```
Room No.  Type            Price/Day     Status
101       Non-AC          1000.00       Available
102       AC              1500.00       Available
103       Non-AC          1200.00       Available
104       AC              2000.00       Available
105       AC              2500.00       Available
Room objects currently stored: 5
```

7.2 Sample Output: Dashboards

```
========== CUSTOMER DASHBOARD ==========
1. View all rooms
2. View available rooms
3. Book a room
4. View my bookings
5. Order food
6. View bill and checkout
7. Logout
```

7.3 Sample Data Files

After a booking, the data files contain lines such as the following (values are examples). In bookings.txt the numbers after the number of days are the quantities of Pizza, Burger, Sandwich, Biryani and Coffee in that order.

```
rooms.txt      101|Non-AC|1000
foods.txt      Pizza|200
customers.txt  rahul|pass123|123456789012
bookings.txt   102|rahul|Rahul Sharma|3|1|2|0|0|3
staff.txt      1|Ramesh|Manager|800
attendance.txt 1|2026-10-27|1
payments.txt   1|2026-10|20800|2026-10-31
```

7.4 Test Cases

No.	Test	Expected result	Observed
1	Enter letters when a number is requested	Error message and the question is repeated	Pass
2	Enter a negative or zero price	Rejected; price must be positive	Pass
3	Book a room that is already booked	“Room is already booked.”	Pass
4	Book a room for 0 days	“Days must be greater than zero.”	Pass
5	Order food for a room that is not booked	“Room is not booked.”	Pass

No.	Test	Expected result	Observed
6	Add a room with an existing number	Duplicate is not added	Pass
7	Register with an existing username	Registration rejected	Pass
8	Type '|' inside a name	Character replaced by '/'; file format stays intact	Pass
9	Close and restart the program	Rooms, foods, customers, bookings and staff data are restored	Pass
10	Damaged line in a data file	Line ignored; program continues	Pass
11	Customer views bookings	Only that customer's bookings are shown	Pass
12	Checkout a booked room	Bill generated, room becomes available	Pass

7.5 Observations

- All menu options work as intended for valid input, and invalid input is handled without crashing the program.
- Data remains available after the program is closed because every change is written to the text files.
- The class design made it easy to add new features, such as the staff module, without changing the room and booking classes.
- The console interface is fast and light, and works on any computer that has a C++ compiler.

GitHub Link

https://github.com/vk9893085252-glitch/Hotel-Mangement-1/tree/main

8. CONCLUSION

The Hotel Management System successfully automates the main activities of a small hotel: managing rooms, registering customers, booking rooms, ordering food, calculating bills, tracking revenue and handling staff attendance and salary payments. The system separates the Admin and Customer roles, validates all input, and stores all information permanently in text files.

From the programming point of view, the project demonstrates a wide range of object-oriented concepts in one practical application. Classes and objects model the real hotel; encapsulation protects the data of each object; abstraction and inheritance are used for the user hierarchy; virtual functions provide runtime polymorphism; function overloading, operator overloading, templates, static members, constructors (including copy and move constructors) and exception handling make the code safe and reusable; and file handling gives persistence.

During development, the team learned how to divide a problem into classes, how to validate data at the point where objects are created, how to design a simple file format and how to test a program with both correct and incorrect input. The project also showed the advantages of the Object -oriented approach: the code is organised, each class can be understood and tested separately, and new features can be added with little change to the existing code. The system fulfils the objectives set at the start and can be used as the foundation of a more advanced hotel software package.

9. SCOPE AND LIMITATIONS

9.1 Scope and Future Enhancements

- **Graphical user interface:** the same classes can be reused with a GUI framework such as Qt to replace the console menus.
- **Database:** text files can be replaced by SQLite or MySQL for faster searching and support for many users at the same time.
- **Security:** store passwords as hashes, keep Aadhaar numbers encrypted and allow the administrator password to be changed.
- **Date-based booking:** store check-in and check-out dates, show room availability for future dates and allow advance reservations.
- **Printable invoices and reports:** generate bills as files and produce daily, weekly or monthly occupancy and revenue reports.
- **Online and mobile access:** connect the system to a web or mobile front end for online booking and payment.
- **Additional services:** laundry, room service, discounts, taxes, loyalty points and customer feedback.

9.2 Limitations

- The interface is text-only, so it is less attractive and less convenient than a graphical interface.
- Passwords and Aadhaar numbers are saved as plain text, and the administrator login is fixed in the source code.
- Each room can store food quantities for at most 100 menu items because the quantity list has a fixed size of 100.
- A booking stores only the number of days and not the actual dates, so future availability cannot be planned.
- Plain text files are suitable only for small amounts of data; they are not designed for many users working at the same time.
- The program has no undo option and no automatic backup of the data files.
- Taxes, discounts and online payment are not part of the current version.

10. REFERENCES

[1] Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley.

[2] Herbert Schildt, *C++: The Complete Reference*, 4th Edition, McGraw-Hill.

[3] E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw-Hill Education.

[4] Robert Lafore, *Object-Oriented Programming in C++*, 4th Edition, Sams Publishing.

[5] Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, *C++ Primer*, 5th Edition, Addison-Wesley.

[6] cppreference.com – C++ language and standard library reference (https://en.cppreference.com).

[7] LearnCpp.com – tutorials on classes, inheritance, virtual functions, templates and exceptions (https://www.learncpp.com).

[8] GeeksforGeeks – C++ programming tutorials and examples (https://www.geeksforgeeks.org).

[9] Lecture notes and laboratory manual of Object-Oriented Programming in C++, Department of Information Technology, Guru Ghasidas Vishwavidyalaya.

11. APPENDIX

Appendix A: Compiling and Running the Program

- Save the source code in a file named *hotel.cpp.*
- Open a terminal in the folder of the file.
- Compile: *g++ hotel.cpp -o hotel -std=c++11*
- Run: *./hotel* (Linux/macOS) or *hotel.exe* (Windows).
- On the first run, the program creates rooms.txt and foods.txt with default data. The other data files are created when the corresponding data is first saved.
- Keep the data files in the same folder as the program; deleting them resets the system to the default data.

Appendix B: Default Data

Default rooms	Default food items
101 – Non-AC – 1000 102 – AC – 1500 103 – Non-AC – 1200 104 – AC – 2000 105 – AC – 2500	Pizza – 200 Burger – 100 Sandwich – 80 Biryani – 180 Coffee – 50

Appendix C: Source Code Excerpts

The following excerpts are taken from the project source file. They show the helper functions, the Room class and the user and staff classes.

C.1 Input and utility functions

```
string readLine(const string& prompt)
    { cout << prompt;
    string value;
    getline(cin >> ws, value);
    return value;
}

double readDouble(const string& prompt)
    { while (true) {
        cout << prompt;
        double value;
        if (cin >> value && value > 0)
            { cin.ignore(numeric_limits<streamsize>::max(),
            '\n'); return value;
        }
        cout << "Enter a number greater than zero.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

string cleanField(string s)
    { replace(s.begin(), s.end(), '|',
    '/'); return s;
}

vector<string> split(const string& line, char delim = '|')
    { vector<string> out;
    string item;
    stringstream ss(line);
    while (getline(ss, item, delim))
        { out.push_back(item);
    }
    return out;
}

template <typename T>
void showMessage(const T& message)
    { cout << message << '\n';
}
```

C.2 Room class (data members, constructors and operator)

```
class Room
{ private:
    int number{};
    string type;
    double price{};
    bool booked{};
    string owner;
    string guest;
    int days{};
    vector<int> foodQty;
    static int objectCount;

public:
    Room() : foodQty(100, 0) { ++objectCount; }

    Room(int n, string t, double p)
        : number(n), type(cleanField(std::move(t))), price(p), foodQty(100, 0)
        { if (n <= 0 || p <= 0 || type.empty()) {
            throw invalid_argument("Invalid room details.");
        }
        ++objectCount;
    }

    Room(const Room& r)
        : number(r.number), type(r.type), price(r.price), booked(r.booked),
          owner(r.owner), guest(r.guest), days(r.days), foodQty(r.foodQty) {
        ++objectCount;
    }

    Room(Room&& r) noexcept
        : number(r.number), type(std::move(r.type)), price(r.price),
          booked(r.booked), owner(std::move(r.owner)), guest(std::move(r.guest)),
          days(r.days), foodQty(std::move(r.foodQty)) {
        ++objectCount;
    }

    ~Room() { --objectCount; }

    void setPrice(double p) {
        if (p <= 0) { throw invalid_argument("Price must be positive."); }
        price = p;
    }

    void release()
        { booked = false;
        owner.clear();
        guest.clear();
        days = 0;
        fill(foodQty.begin(), foodQty.end(), 0);
    }

    static int getObjectCount() { return objectCount; }

    friend ostream& operator<<(ostream& os, const Room& r) {
        os << left
           << setw(10) << r.number
           << setw(16) << r.type
           << setw(14) << fixed << setprecision(2) << r.price
           << (r.booked ? "Booked" : "Available");
        return os;
    }
};

int Room::objectCount = 0;
```

C.3 FoodItem, User, Admin, Customer and Staff

```
class FoodItem
{ private:
    string name;
    double price{};
public:
    FoodItem(string n, double p) : name(cleanField(std::move(n))), price(p)
        { if (name.empty() || p <= 0) {
            throw invalid_argument("Food name and positive price are required.");
        }
    }
    string getName() const { return name; }
    double getPrice() const { return price; }
};

class Admin : public User
{ public:
    Admin() : User("admin", "admin123") {}
    void dashboard() const override { /* prints the 11 admin options */ }
};

class Customer : public User
{ private:
    string aadhaarNumber;
public:
    Customer(string u, string p, string aadhaar = "")
        : User(std::move(u), std::move(p)), aadhaarNumber(std::move(aadhaar)) {}
    string getAadhaarNumber() const { return aadhaarNumber; }
    void dashboard() const override { /* prints the 7 customer options */ }
};

class Staff
{ private:
    int id{};
    string name, role;
    double dailyWage{};
public:
    Staff(int i, string n, string r, double w)
        : id(i), name(cleanField(std::move(n))), role(cleanField(std::move(r))), dailyWage(w)
        { if (id <= 0 || name.empty() || role.empty() || w <= 0) {
            throw invalid_argument("Invalid staff details.");
        }
    }
};

struct Attendance { int staffID; string date; bool present; };
struct Payment    { int staffID; string period; double amount; string date; };
```

C.4 Hotel class: overloaded search and saving

```
// Function overloading: search by room number
int findRoom(int number) const {
    for (size_t i = 0; i < rooms.size(); ++i)
        if (rooms[i].getNumber() == number) return static_cast<int>(i);
    return -1;
}

// Function overloading: search available room by type
int findRoom(const string& type) const {
    for (size_t i = 0; i < rooms.size(); ++i)
        if (rooms[i].getType() == type && !rooms[i].isBooked())
            return static_cast<int>(i);
    return -1;
}

void saveRooms() const
    { ofstream
    f(ROOMS_FILE);
    if (!f) { throw runtime_error("Cannot save rooms."); }
    for (const auto& r : rooms) {
        f << r.getNumber() << '|' << r.getType() << '|' << r.getPrice() << '\n';
    }
}

bool paidFor(int id, const string& period) const
    { for (const auto& p : payments)
        if (p.staffID == id && p.period == period) return true;
    return false;
```